University Name

Department of Computer Science

AI-Powered Medical Appointment and Disease Risk Prediction System

Project Documentation Report

Submitted in partial fulfillment of the requirements for academic project evaluation.

Submitted By	Student Name
Roll Number	Roll Number
Guide Name	Guide Name
Guide Designation	Project Guide
Academic Year	2025-2026
Prepared On	25 March 2026

A complete implementation report covering architecture, data, modules, workflows, testing, and outcomes.

Declaration by the Student

I hereby declare that the project work entitled "AI-Powered Medical Appointment and Disease Risk Prediction System" submitted to University Name, in partial fulfillment of the requirements for the award of the degree, is a record of original work carried out by me under the guidance of Guide Name. This work has not been submitted elsewhere for any other degree or diploma.

Student Signature	_____
Name	Student Name
Roll Number	Roll Number
Date	_____

Certificate

This is to certify that the project report entitled "AI-Powered Medical Appointment and Disease Risk Prediction System" has been completed by Student Name (Roll Number) under my guidance and supervision. The work presented is found satisfactory and is approved for submission.

Guide Signature	_____
Guide Name	Guide Name
Designation	Project Guide
Head of Department Signature	_____

Acknowledgment

I express my sincere gratitude to my project guide, faculty members, and department for their guidance and encouragement throughout this work. I also thank my friends and family for their constant support, and I acknowledge the open-source community whose tools and documentation helped in the successful development of this project.

Abstract

The AI-Powered Medical Appointment and Disease Risk Prediction System is a web-based healthcare application designed to streamline patient intake, appointment booking, and early disease risk assessment. The project addresses delays in manual triage by combining a trained Decision Tree model with rule-based guardrails so that high-risk cases can receive faster clinical attention. The implementation uses a hybrid FastAPI and Flask architecture: FastAPI exposes API and compatibility routes, while Flask handles the role-based portal, session management, dashboards, and patient-facing workflows. HTML/CSS/JavaScript templates provide the interface layer, CSV and SQLite files provide operational storage, and an optional OpenAI-backed patient chat can answer both contextual portal questions and general guidance prompts. Patients can register, submit health details, review risk summaries, chat with the assistant, book appointments, cancel appointments, and inspect booking history. Doctors and nurses receive dedicated dashboards to review cases, manage queue flow, update records, track leave dates, and apply emergency-availability rules. The trained model uses 25 engineered features from a dataset of 1455 usable records, and the current evaluation reproduces a test accuracy of 98.97%. The live project snapshot currently includes 21 patient profiles, 9 appointments, 8 doctor profiles, and 25 rendered portal templates. The project demonstrates how AI can support faster decision-making in appointment management while remaining practical for small clinical environments. Future scope includes auth consolidation, stronger deployment security, notification services, richer explainability, and larger clinical datasets.

Table of Contents

Declaration by the Student 2

Certificate 3

Acknowledgment 4

Abstract 5

Table of Contents 6

Chapter 1: Introduction 8

1.1 Overview 8

1.2 Problem Statement 8

1.3 Objectives 8

1.4 Scope 8

1.5 Limitations and Assumptions 8

Chapter 2: Literature Review / Existing System 9

2.1 Existing System 9

2.2 Limitations in Existing Approaches 9

2.3 Gap Analysis 9

Chapter 3: System Analysis 10

3.1 Functional Requirements 10

3.2 Non-Functional Requirements 10

3.3 Feasibility Study 10

3.4 System Environment 10

3.5 Use Case / Flow Overview 11

Chapter 4: System Design 12

4.1 Architectural Design 12

4.2 Data Flow Design 12

4.3 Database Design 12

4.4 Current Stored Project Data Snapshot 13

Chapter 5: Implementation 14

5.1 Tools and Technologies Used 14

5.2 Modules / Components 14

5.3 Implementation Highlights 14

5.4 Security, Validation, and Data Handling 15

Chapter 6: Testing 16

6.1 Testing Approach 16

6.2 Sample Test Cases 16

6.3 Model Evaluation 16

Chapter 7: Results and Discussion 18

7.1 Dataset and Feature Summary 18

7.2 Output Discussion 18

7.3 Operational Coverage Snapshot 18

7.4 Performance Visualizations 18

Chapter 8: Conclusion and Future Work 21

8.1 Conclusion 21

8.2 Future Work 21

References / Bibliography 22

Appendices 23

Appendix A: Installation Guide 23

Appendix B: Major Routes and Features 23

Appendix C: Project Artifacts 23

Chapter 1: Introduction

1.1 Overview

Healthcare systems frequently face delays when patients must manually describe symptoms and wait for administrative prioritization before a doctor reviews the case. This project proposes a single digital workflow where patient information, risk assessment, and appointment scheduling are combined into one coordinated platform.

1.2 Problem Statement

Traditional appointment booking and triage processes often depend on manual judgment, fragmented records, and repeated data entry. As a result, urgent patients may not be identified quickly, doctors may not get structured case summaries in time, and nurses may have limited support for queue management.

1.3 Objectives

• Provide patient, doctor, and nurse portals in a single integrated healthcare workflow.

• Combine FastAPI API endpoints with Flask-rendered role-based portal pages.

• Predict disease risk level from patient symptoms and vital features using an ML model with safety guardrails.

• Automatically derive appointment priority, doctor availability decisions, and emergency escalation guidance.

• Store patient details, appointments, leave records, and review notes in a persistent data layer.

• Offer a practical base for future intelligent healthcare automation and deployment hardening.

1.4 Scope

The current implementation targets clinic-style workflow support. It covers patient registration, health-detail submission, disease risk prediction, appointment booking, doctor review, nurse queue handling, doctor leave management, emergency-doctor availability checks, patient booking history and cancellation, contextual patient chat, and compatibility API routes for prediction and booking operations.

1.5 Limitations and Assumptions

• The model is trained on a structured dataset rather than live hospital EHR data.

• The training snapshot is dominated by Low and Medium classes, although runtime guardrails can escalate cases to High Risk.

• Patient account handling currently spans both a portal registry CSV and a separate hashed API-auth CSV, and would benefit from consolidation.

• The application is suitable for academic or prototype use and would need stronger compliance controls for production healthcare deployment.

• Guide, student, and university details on the cover page are placeholders and can be updated easily.

Chapter 2: Literature Review / Existing System

2.1 Existing System

Conventional appointment systems primarily focus on slot booking and usually leave triage as a manual task. Many small-scale systems also separate patient records, booking, and doctor review into different tools, which reduces efficiency and traceability.

2.2 Limitations in Existing Approaches

• Urgent patients can be overlooked when prioritization is fully manual.

• Existing systems often lack integrated predictive analytics.

• Administrative staff may re-enter the same information in multiple places.

• Role-specific dashboards are often absent in basic booking platforms.

2.3 Gap Analysis

The proposed project fills the gap by linking health data capture, machine learning prediction, rule-based triage, and appointment allocation in one application. The result is a more structured and intelligent flow that benefits both patients and clinical staff.

Chapter 3: System Analysis

3.1 Functional Requirements

Requirement	Description
Patient registration and login	Create accounts and access the patient workflow securely.
Health data entry	Capture age, symptoms, glucose, BMI, blood pressure, sleep, and history.
Risk prediction	Generate risk label, confidence breakdown, and guidance from the model.
Appointment booking and cancellation	Book appointments, assign priority, and manage booking history.
Doctor dashboard	Review appointments, risk details, and patient database information.
Nurse queue dashboard	Inspect patient queue and update records before doctor review.
Doctor availability / leave	Track leave dates and expose doctor availability status for scheduling decisions.
Emergency scheduling support	Enforce emergency-doctor-only windows and recommend alternatives for high-risk cases.
AI chat support	Provide contextual help for patient-side questions when configured.

3.2 Non-Functional Requirements

Category	Expectation
Usability	Simple role-based pages and guided health-detail forms.
Performance	Fast prediction and scheduling response for individual patient requests.
Reliability	Persistent storage using CSV files and SQLite tables with route-level validation.
Security	Session guards, password policies, hashed doctor/nurse credentials, and environment-controlled s
Maintainability	Modular backend files for API entry, portal routes, prediction, triage, auth, and data access.

3.3 Feasibility Study

• Technical feasibility: achieved using Python, FastAPI, Flask, pandas, scikit-learn, and SQLite.

• Economic feasibility: suitable for academic implementation with low infrastructure cost.

• Operational feasibility: supports clinic-style users through separate patient, doctor, and nurse flows with lightweight file-backed storage.

3.4 System Environment

Item	Details
Programming Language	Python

Item	Details
Web Frameworks	FastAPI + Flask
Serving Layer	Uvicorn with WSGIMiddleware for portal mounting
Frontend	HTML, CSS, JavaScript templates
Data/Storage	CSV files and SQLite
ML Libraries	scikit-learn, pandas, numpy, joblib
Validation	Pydantic
Optional AI Service	OpenAI-based patient chat when API key is configured
Platform	Web application

3.5 Use Case / Flow Overview

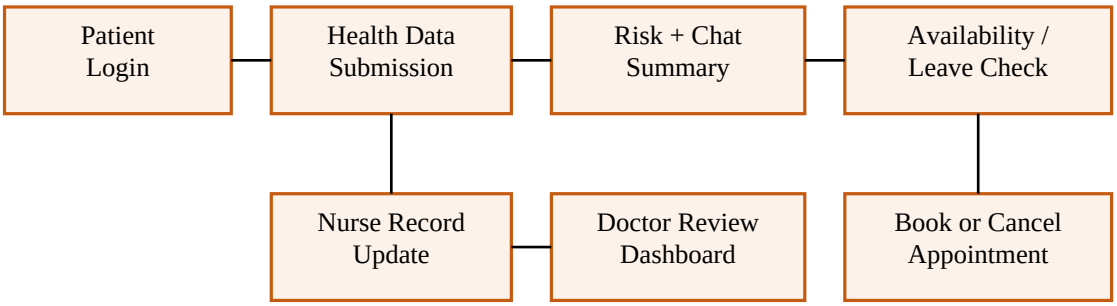

Figure 1: High-level process flow of the proposed system.

Chapter 4: System Design

4.1 Architectural Design

The application follows a layered hybrid architecture. FastAPI acts as the API-facing entry layer, serves static assets, exposes prediction and booking endpoints, and mounts the Flask portal under a shared route surface. Flask coordinates server-rendered templates, role-based sessions, dashboard workflows, booking, cancellation, chat orchestration, and staff review actions. The prediction layer uses a Decision Tree pipeline with rule-based triage and emergency/availability logic. Persistent operational data is managed through CSV-based account and scheduling files plus an SQLite database for patient profiles and appointments.

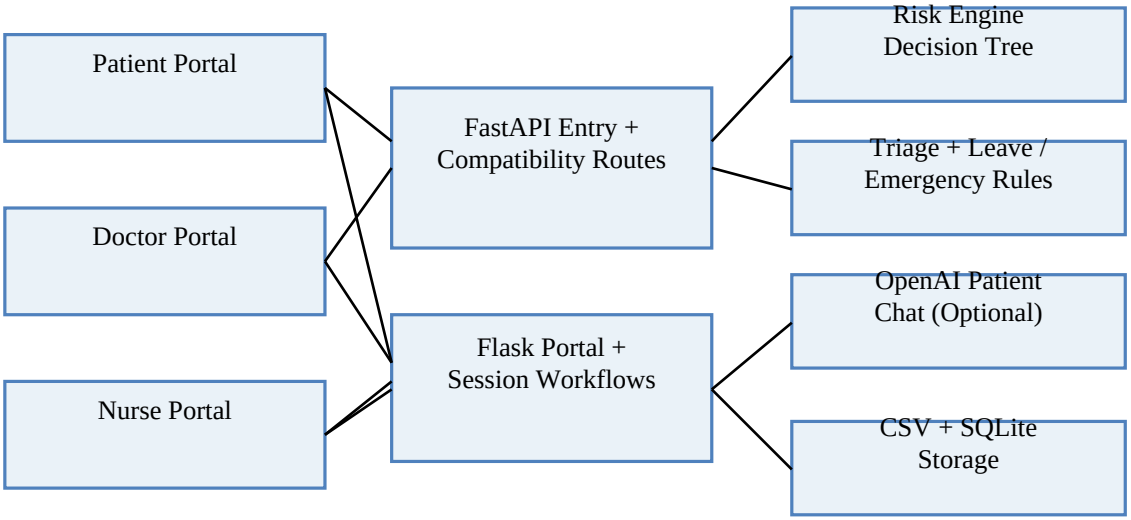

Figure 2: System architecture of the application.

4.2 Data Flow Design

• Patient data flows from form submission into feature normalization and persistence.

• Normalized features are passed to the prediction engine for risk computation.

• Risk level is translated into appointment priority by the triage module.

• Appointment requests are checked against doctor leave data and emergency-doctor scheduling rules.

• Appointments and profiles are stored for doctor and nurse dashboard access.

• Patient chat uses portal context first and optionally calls the OpenAI client for broader responses.

4.3 Database Design

Data Store	Purpose
patients.csv	Portal patient registry, names, unique codes, and health-details completion flag

Data Store	Purpose
patient_accounts.csv	Separate hashed patient credential store used by the API auth manager
doctor_accounts.csv	Doctor authentication credentials
nurse_accounts.csv	Nurse authentication credentials
doctor_profiles.csv	Doctor metadata, specialization, available time, and emergency flag
doctor_leave.csv	Doctor leave dates used by scheduling checks
new_patient_data.csv	Latest submitted patient features plus nurse/doctor review fields
patient_records.db / patient_profiles	Structured health details and review history
patient_records.db / appointments	Booked appointments, priorities, and stored prediction output

4.4 Current Stored Project Data Snapshot

Artifact	Current Count
Portal patient registry rows	23
API patient auth rows	0
Doctor accounts	8
Nurse accounts	2
Doctor profiles	8
Emergency doctor profiles	2
Doctor leave records	5
New patient health records	21
SQLite patient profiles	21
SQLite appointments	9
HTML templates	25
Static assets	5

Chapter 5: Implementation

5.1 Tools and Technologies Used

Layer	Technology
Frontend	HTML templates, CSS, JavaScript
API / Entry Layer	FastAPI, Uvicorn, WSGIMiddleware
Portal Layer	Flask application with role-based routes and sessions
Machine Learning	Decision Tree pipeline with preprocessing and rule guardrails
Storage	CSV datasets and SQLite database
AI Integration	OpenAI Responses API for optional patient chat
Utilities	pandas, numpy, joblib, pydantic

5.2 Modules / Components

Module	Purpose
backend/main.py	FastAPI entrypoint, API routes, static mount, portal mount, and compatibility redirects
backend/flask_app.py	Main portal workflows, routing, sessions, dashboards, booking, cancellation, and chat orc
backend/predict.py	Feature normalization, model prediction, and response shaping
backend/triage.py	Maps risk levels to appointment priority and slot recommendation
backend/patient_db.py	SQLite schema and patient/appointment persistence
backend/patient_auth.py	Separate patient auth manager with hashed credentials for API flows
backend/doctor_auth.py	Doctor signup and login with password hashing and validation
backend/nurse_auth.py	Nurse signup and login workflow
backend/paths.py	Environment-aware dataset paths and storage configuration
backend/init_datasets.py	Bootstrap script for required dataset CSV files
frontend/templates	Patient, doctor, and nurse interface pages
frontend/static	Shared assets, icons, and page animation files

5.3 Implementation Highlights

- FastAPI mounts the Flask portal and preserves legacy compatibility routes for earlier URLs.
- Portal routes support patient, doctor, and nurse role separation with session-based access guards.

• A trained model and rule guardrails work together to produce Low, Medium, or High risk output.

• Appointments include priority labels such as Immediate, Same Day, and Normal.

• Booking logic validates doctor leave data and enforces emergency-doctor-only night scheduling when required.

• Patient booking confirmation pages expose upcoming and past appointments, and allow appointment cancellation.

• Stored prediction data is accessible from dashboard workflows for follow-up actions.

• The patient portal can optionally use an OpenAI-backed chatbot with patient-context prompting.

5.4 Security, Validation, and Data Handling

• Role-specific decorators protect patient, doctor, and nurse routes from cross-role access.

• Doctor and nurse credentials use salted PBKDF2 hashes, and a separate API patient auth manager also supports hashed patient credentials.

• Environment variables control dataset paths, Flask session security flags, and optional OpenAI settings.

• Patient data updates are synchronized to CSV storage and SQLite profile records for downstream dashboards.

Chapter 6: Testing

6.1 Testing Approach

• Functional testing of patient signup, login, and appointment workflow.

• Prediction testing using the trained Decision Tree model on held-out data.

• Dashboard testing for doctor and nurse visibility into queue and records.

• Scheduling-rule testing for leave handling and emergency-doctor availability checks.

• Persistence testing for CSV and SQLite updates after user actions.

• Patient chat testing for contextual replies and optional OpenAI fallback behavior.

6.2 Sample Test Cases

Test Case	Expected Result	Observed Outcome
Patient submits health details	Profile saved and prediction-ready features created	Completed
Risk prediction request	Risk label, probability, and guidance returned	Completed
Appointment submission	Appointment stored with priority and recommended slot	Completed
Night-time booking with non-emergency doctor	System blocks booking and suggests emergency-capable alternatives	Completed
Doctor marked on leave	System reports doctor unavailable and returns next-step guidance	Completed
Doctor dashboard access	Doctor sees prioritized appointments and patient records	Completed
Nurse queue review	Nurse can view and update patient intake details	Completed
Appointment cancellation	Selected appointment is removed from patient history	Completed

6.3 Model Evaluation

Metric	Value
Usable records	1455
Training records	1164
Testing records	291
Test accuracy	98.97%
Precision	98.99%
Recall	98.97%
F1-score	98.97%

Metric	Value
Confusion matrix	[[168, 0], [3, 120]]

Chapter 7: Results and Discussion

7.1 Dataset and Feature Summary

Item	Value
Clean dataset shape	1530 rows x 9 columns
Feature-engineered dataset shape	1530 rows x 28 columns
Records used for training/evaluation	1455
Dropped rows due to missing values	75
Model input feature count	25
Risk distribution	Low: 838, Medium: 617

7.2 Output Discussion

The system demonstrates strong predictive consistency on the available dataset and supports a workflow where model output is immediately usable for operational appointment decisions. The current dataset snapshot is dominated by Low and Medium risk patterns, and the resulting model performance reflects this distribution. Beyond classification, the project adds operational value through leave-aware scheduling, emergency-doctor recommendation, appointment history management, nurse review persistence, doctor dashboards, and patient-context chat support.

7.3 Operational Coverage Snapshot

Operational Item	Value
Portal patient registry rows	23
API patient auth rows	0
Doctor profiles / emergency doctors	8 / 2
Doctor leave rows	5
Stored patient profiles / appointments	21 / 9
Portal templates / static assets	25 / 5

7.4 Performance Visualizations

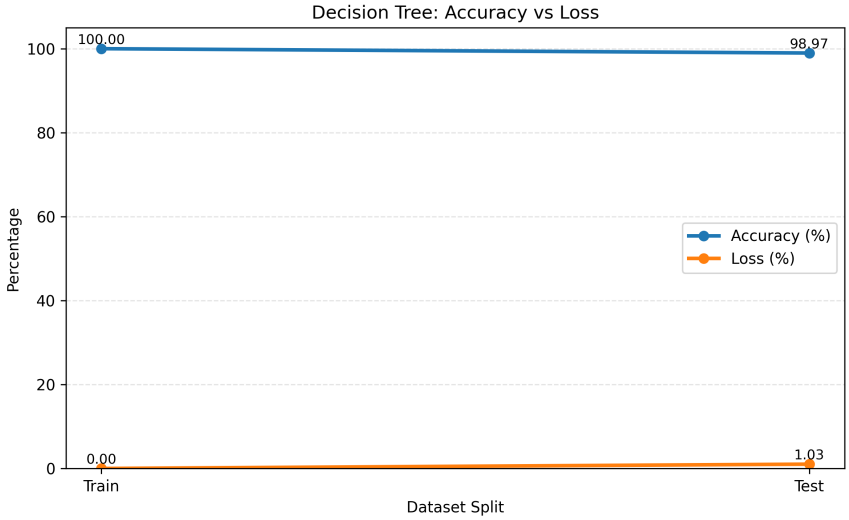

Figure 3: Decision Tree accuracy versus loss.

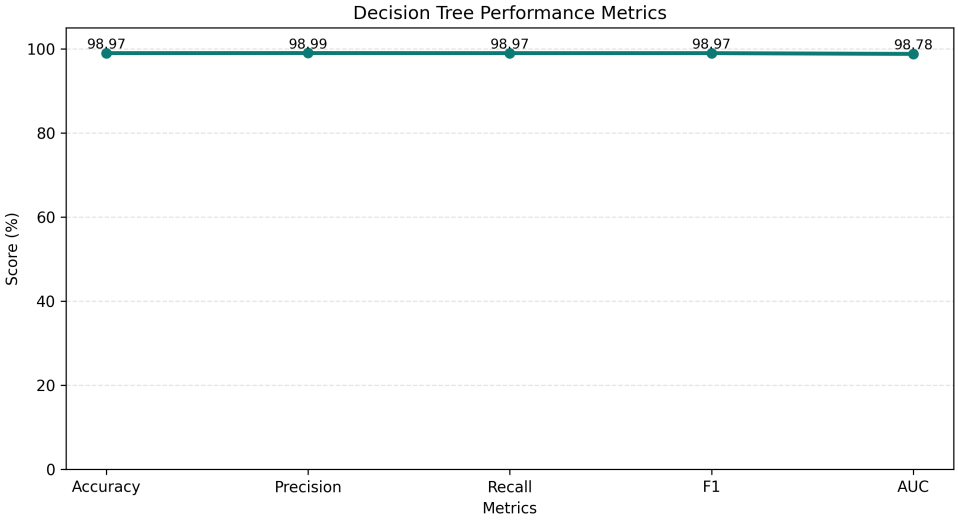

Figure 4: Decision Tree performance metrics.

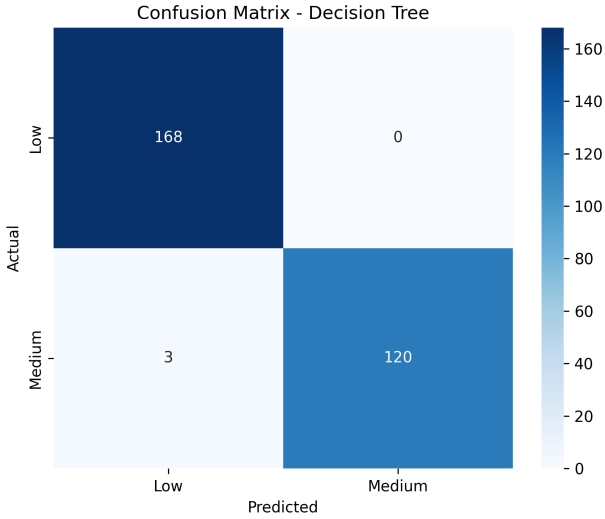

Figure 5: Confusion matrix of the trained model.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

The AI-Powered Medical Appointment and Disease Risk Prediction System successfully integrates predictive analytics with a practical healthcare workflow. The project combines a hybrid FastAPI and Flask architecture, role-based access, risk assessment, triage, leave-aware appointment management, emergency scheduling logic, patient-context chat, and persistent storage in one working web application. The outcome is a strong academic prototype that demonstrates the value of AI-assisted decision support in healthcare operations.

8.2 Future Work

• Use larger and clinically richer datasets to improve generalization.

• Expand the classifier to support more risk classes and disease categories.

• Add explainable AI views for doctor-facing decision support.

• Unify the separate patient-auth and portal-registry flows into one consistent authentication model.

• Integrate notification services for reminders and emergency escalation.

• Strengthen production readiness with audit logging and healthcare compliance measures.

References / Bibliography

- FastAPI Documentation.
- Flask Documentation.
- pandas Documentation.
- scikit-learn Documentation.
- SQLite Documentation.
- Uvicorn and ASGI integration references.
- OpenAI API platform documentation for optional chatbot integration.
- Project source code, templates, datasets, and trained artifacts in the current workspace.

Appendices

Appendix A: Installation Guide

• Create and activate a Python virtual environment.

• Install project dependencies used by the FastAPI entry layer, Flask portal, and ML pipeline.

• Initialize dataset CSV files using the dataset bootstrap script when needed.

• Configure environment variables such as CSV paths and optional OpenAI settings.

• Run the project through backend/main.py so both API and portal routes are available.

Appendix B: Major Routes and Features

• FastAPI entry routes cover predict-risk, book-appointment, patient-features lookup, appointments listing, static serving, and portal mounting.

• Patient routes cover signup, login, health details, AI chat, booking, booking confirmation, cancellation, feature updates, and emergency-doctor discovery.

• Doctor routes cover signup, login, dashboard, appointments, patient database, leave management, availability status, and risk review.

• Nurse routes cover signup, login, dashboard, appointment queue, patient-record updates, and risk review.

• Compatibility routes preserve earlier URLs by redirecting them into the shared portal flow.

Appendix C: Project Artifacts

• Training and preprocessing artifacts include Healthcare.csv, Healthcare_Cleaned.csv, Healthcare_FeatureEngineered.csv, the trained model, and the label encoder.

• Evaluation artifacts include the accuracy/loss chart, metrics summary, and confusion matrix images bundled in the backend directory.

• Operational datasets under backend/datasets store patient registry rows, auth files, doctor profiles, leave records, health submissions, and SQLite appointment data.